

Course 3 – Identity Cloud Building Blocks

Module 1 – Authentication Using Single Sign-On (SSO)

Authentication Using Single Sign-On (SSO)

Authentication is the first security step performed before a user is allowed to access the Saviynt Identity Cloud.

Organizations generally do not want users to remember separate usernames and passwords for every application. Instead, they use **Single Sign-On (SSO)** so that users authenticate once and access multiple enterprise applications securely.

Saviynt supports **SAML-based Single Sign-On**, allowing it to integrate with Identity Providers such as **Okta, Azure AD (Microsoft Entra ID), Ping Identity, Google Workspace, ADFS**, etc.

What is Authentication?

Definition

Authentication is the process of **verifying the identity of a user or system** before granting access.

Authentication answers the question:

"Who are you?"

Only after successful authentication does the system know the user is genuine.

Example

Rahul opens the Saviynt URL.

He enters his company credentials.

The system verifies:

- Username
- Password
- MFA (if enabled)

If everything is correct, Rahul is authenticated.

Authentication Factors

Authentication can use one or more factors.

1. Something You Know

- Password
- PIN
- Security Question

Example:

Password = Rahul@123

2. Something You Have

- Mobile Phone
- Hardware Token
- Smart Card

Example:

OTP received on mobile.

3. Something You Are

Biometric authentication.

Examples:

- Fingerprint
- Face Recognition
- Iris Scan

Modern organizations usually combine two or more factors.

This is called **Multi-Factor Authentication (MFA)**.

What is Authorization?

Definition

Authorization determines **what a user is allowed to do after authentication**.

It answers:

"What are you allowed to access?"

Example

Rahul successfully logs into Saviynt.

Authentication is complete.

Now Saviynt checks his SAV Role.

Rahul can:

- Request Access
- View Reports

But cannot:

- Configure SSO
- Delete Applications

Those actions require administrator permissions.

Authentication vs Authorization

Authentication	Authorization
Verifies identity	Verifies permissions
Happens first	Happens after authentication
Answers "Who are you?"	Answers "What can you do?"
Uses username, password, MFA	Uses roles and permissions
Example: Login	Example: Access Dashboard

Easy Way to Remember

Authentication = Login

Authorization = Permission

What is Single Sign-On (SSO)?

Definition

Single Sign-On (SSO) is an authentication mechanism that allows users to log in **once** and access multiple applications without logging in again.

The user authenticates only with the Identity Provider.

Other applications trust that authentication.

Real-World Example

Suppose your company uses:

- Outlook
- Microsoft Teams
- Saviynt
- Salesforce
- Workday

Without SSO:

You log in separately to every application.

With SSO:

You log into Okta once.

Then you can access all applications without entering credentials again.

Why Organizations Use SSO

Large organizations have hundreds of applications.

Without SSO:

- Users remember many passwords.
- Password reset tickets increase.
- Users reuse passwords.
- Security decreases.

SSO solves these problems.

Benefits of SSO

Improved User Experience

One login provides access to multiple applications.

Better Security

Centralized authentication.

Supports MFA.

Reduces password reuse.

Reduced Helpdesk Tickets

Fewer password reset requests.

Centralized Identity Management

User authentication is controlled from one Identity Provider.

Faster Access

Employees access applications quickly.

Better Compliance

Authentication logs are centralized.

Easy to audit.

Native Login vs SSO

Native Login

Authentication is handled directly by Saviynt.

User enters:

- Username
- Password

Saviynt validates credentials.

Advantages

- Simple setup
- No external dependency

Disadvantages

- Separate password
 - Poor user experience
 - Harder to manage
-

Single Sign-On

Authentication is handled by an external Identity Provider.

Examples:

- Okta
- Azure AD
- Google
- Ping Identity

Saviynt trusts the Identity Provider.

Advantages

- One login
- Better security
- Centralized authentication
- Supports MFA

Disadvantages

- Requires SAML configuration

- Depends on Identity Provider availability
-

Service Provider (SP)

Definition

A Service Provider is the application that users want to access.

It trusts the Identity Provider to authenticate users.

In this course,

Saviynt acts as the Service Provider (SP).

SP is responsible for:

- Redirecting users to IdP
 - Receiving SAML Response
 - Validating Assertions
 - Authorizing users
-

Identity Provider (IdP)

Definition

The Identity Provider authenticates users and confirms their identity.

Examples:

- Okta
- Azure AD
- Google Workspace
- Ping Identity
- ADFS

The IdP stores:

- User accounts
 - Passwords
 - MFA configuration
-

SP vs IdP

Service Provider (SP)	Identity Provider (IdP)
Provides application	Provides authentication
Saviynt	Okta
Trusts IdP	Authenticates users
Grants application access	Verifies credentials

Easy example:

Saviynt = Office Building

Okta = Security Guard

Security Guard verifies identity.

Office Building allows entry.

What is SAML?

Definition

Security Assertion Markup Language (SAML) is an XML-based standard used to exchange authentication and authorization information between an Identity Provider and a Service Provider.

SAML is the protocol used for SSO in Saviynt.

Main Components of SAML

SAML Request



Identity Provider



Authentication



SAML Assertion



SAML Response



Service Provider



User Login

SAML Request

Definition

A SAML Request is generated by the Service Provider.

Purpose:

Ask the Identity Provider to authenticate the user.

Generated by:

Saviynt

Contains:

- Application URL
 - Request ID
 - Issuer
 - Destination
-

SAML Response

Definition

A SAML Response is generated by the Identity Provider after successful authentication.

Generated by:

Okta

Contains:

- User Identity
- Authentication Status
- User Attributes
- Digital Signature
- SAML Assertion

Saviynt validates this response.

What is a SAML Assertion?

Definition

A SAML Assertion is an XML document that contains authenticated user information.

It proves that the user has already been authenticated.

Typical attributes include:

- Username
- Email
- First Name
- Last Name
- Employee ID
- Groups

Saviynt uses these attributes to identify the user.

X.509 Certificate

Definition

An X.509 Certificate is a digital certificate used to verify the identity of systems and securely sign SAML messages.

Purpose:

- Encryption
- Digital Signature
- Trust
- Integrity

Saviynt provides a default Service Provider certificate, or organizations can upload their own certificate issued by a Certificate Authority (CA).

Metadata XML

Definition

Metadata XML is a configuration file exchanged between the Service Provider and Identity Provider.

It contains configuration details such as:

- Entity ID
- SSO URL

- Logout URL
- X.509 Certificate
- Endpoints
- Supported Bindings

It allows both systems to trust each other.

SP-Initiated Flow

Definition

The login process starts from the Service Provider (Saviynt).

Workflow

1. User opens Saviynt URL.
 2. Saviynt generates a SAML Request.
 3. Browser redirects to Okta.
 4. User authenticates.
 5. Okta generates a SAML Response.
 6. Browser sends response to Saviynt.
 7. Saviynt validates the response.
 8. User enters Saviynt.
-

IdP-Initiated Flow

Definition

The login process starts from the Identity Provider.

Workflow

1. User logs into Okta.
2. User clicks the Saviynt application tile.
3. Okta immediately generates a SAML Response.
4. Browser sends the response to Saviynt.

5. Saviynt validates it.
6. User is logged in.

No SAML Request is generated because the process starts at the IdP.

Complete Authentication Workflow

1. User opens the Saviynt URL.
 2. Saviynt (SP) creates a SAML Authentication Request.
 3. Browser redirects the request to Okta (IdP).
 4. Okta authenticates the user using password and MFA.
 5. Okta generates a signed SAML Assertion.
 6. Assertion is included inside a SAML Response.
 7. Browser sends the SAML Response to Saviynt.
 8. Saviynt validates:
 - Digital Signature
 - Certificate
 - Assertion
 - Timestamp
 9. Saviynt searches for the user in its repository.
 10. If the user exists, authorization is performed based on the assigned SAV Role.
 11. The Saviynt Home Page is displayed.
 12. If the user does not exist or validation fails, access is denied.
-

Complete Okta Configuration (High-Level)

1. Configure Saviynt as the **Service Provider (SP)**.
2. Use the default SP certificate or upload your organization's X.509 certificate.
3. Configure general SP settings (Entity ID, ACS URL, Logout URL).
4. Create a new SAML application in Okta.
5. Enter the Saviynt Single Sign-On URL.
6. Download the Okta Metadata XML file.
7. Import the Metadata XML into Saviynt under Identity Provider settings.
8. Save the configuration in Draft Mode.
9. Activate the SSO configuration.
10. Test the login using the Saviynt URL.
11. Verify that authentication redirects to Okta and successfully returns to Saviynt.

Troubleshooting SSO

Common checks when SSO does not work:

- Verify the Metadata XML file.
 - Check whether the X.509 certificate is expired.
 - Ensure Entity IDs match.
 - Verify the Assertion Consumer Service (ACS) URL.
 - Confirm NameID mapping is correct.
 - Check user attributes (email, username).
 - Verify system time (clock skew).
 - Review Saviynt and IdP logs.
-

Common Errors

Invalid SAML Response

Cause:

Response has been modified or is malformed.

Invalid Signature

Cause:

Wrong or expired X.509 certificate.

User Not Found

Cause:

Authenticated user does not exist in the Saviynt repository.

NameID Mismatch

Cause:

The attribute sent by the IdP does not match the expected username in Saviynt.

Metadata Mismatch

Cause:

Incorrect Entity ID, ACS URL, or Metadata XML configuration.

Certificate Expired

Cause:

The signing certificate has expired.

Solution:

Replace the certificate and update the metadata.

Clock Skew Error

Cause:

The system clocks of the IdP and SP are not synchronized.

Solution:

Synchronize both servers using NTP.

Best Practices

- Always enable **Sign Assertions** to ensure message integrity.
 - Prefer **SSO with MFA** over native authentication for stronger security.
 - Keep X.509 certificates valid and monitor their expiration dates.
 - Use **Draft Mode** to test SSO configuration before activation.
 - Validate SSO in a test environment before deploying to production.
 - Restrict who can modify SSO settings.
 - Regularly review SAML logs for troubleshooting and auditing.
-

Quick Revision

- Authentication verifies identity.
- Authorization determines permissions.
- SSO enables one login for multiple applications.
- Saviynt acts as the **Service Provider (SP)**.
- Okta acts as the **Identity Provider (IdP)**.
- SAML is the protocol used for SSO.
- SAML Request is generated by the SP.
- SAML Response is generated by the IdP.
- Assertion contains authenticated user information.
- Metadata XML establishes trust between SP and IdP.
- X.509 certificates provide signing, encryption, and trust.
- SP-Initiated Flow starts from Saviynt.
- IdP-Initiated Flow starts from Okta.
- After successful authentication, Saviynt authorizes the user based on their assigned SAV Role.

This diagram shows the **SAML Authentication Flow** between three components:

- **Identity Provider (IdP)** → Authenticates the user (e.g., Okta, Azure AD, Ping)
- **End User Browser (Agent)** → User's web browser (Chrome, Edge)
- **Service Provider (SP)** → Application the user wants to access (Saviynt)

Let's understand every step in simple words.

SAML Authentication Flow

Components

+-----+	+-----+	+-----+
Identity Provider	End User Browser	Service Provider
(Okta/Azure AD)	(Chrome/Edge)	(Saviynt)
+-----+	+-----+	+-----+

Step 1: User Accesses the Service Provider

Image: Step 1



What happens?

The user enters the Saviynt URL in the browser.

Example:

<https://company.saviyntcloud.com>

The browser sends a request to Saviynt.

Example

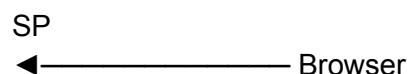
Rahul opens

<https://saviynt.company.com>

Saviynt receives the request.

Step 2: Service Provider Redirects User to Identity Provider

Image: Step 2



(SP sends a SAML Authentication Request to the browser, which is then redirected to the IdP.)

What happens?

Saviynt does **not** ask for the username and password.

Instead, it says:

"Go to Okta and authenticate yourself."

Saviynt creates a **SAML Authentication Request**.

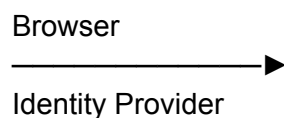
The browser carries this request to the Identity Provider.

Why?

Because authentication is handled by Okta, not by Saviynt.

Step 3: Browser Sends SAML Request to Identity Provider

Image: Step 3



What happens?

The browser forwards the SAML request received from Saviynt to Okta.

Okta now knows:

- Which application is requesting authentication (Saviynt)
 - Which Service Provider sent the request
-

Step 4: Identity Provider Authenticates the User

Image: Step 4

Identity Provider



Authenticate User

What happens?

Okta asks for the user's credentials.

Example:

Username: rahul

Password: *****

Or, if the user is already logged into Okta, this step is skipped.

Okta verifies:

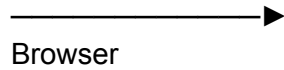
- Username
- Password
- Multi-Factor Authentication (if enabled)

If everything is correct, the user is authenticated.

Step 5: Identity Provider Generates SAML Assertion

Image: Step 5

Identity Provider



What happens?

After successful authentication, Okta creates a **SAML Assertion**.

A **SAML Assertion** is an XML document that proves:

"This user has already been authenticated."

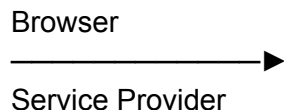
It usually contains:

- Username
- Email
- Employee ID
- Department
- Roles
- Authentication time

It is **digitally signed** using the IdP's certificate so Saviynt can verify that it has not been tampered with.

Step 6: Browser Sends SAML Assertion to Service Provider

Image: Step 6



What happens?

The browser sends the SAML Assertion received from Okta to Saviynt.

Saviynt validates:

- Digital signature

- Certificate
- Expiration time
- Issuer
- Audience

If all checks pass, the assertion is accepted.

Step 7: Service Provider Creates a Session

Image: Step 7

SP

↓

Creates Security Session

↓

Browser

What happens?

Saviynt now trusts the authenticated user.

It creates a session and sends a **session cookie** to the browser.

This cookie allows the user to continue using Saviynt without logging in again during the session.

Step 8: Browser Requests the Resource

Image: Step 8

Browser

—————▶
Service Provider

What happens?

Now that the browser has a valid session cookie, it requests the actual page or resource.

Example:

Dashboard

Access Requests

Reports

Step 9: Service Provider Responds with the Requested Resource

Image: Step 9

Service Provider
—————▶
Browser

What happens?

Saviynt sends the requested page back to the browser.

The user now sees the Saviynt Home Page and can work according to their assigned permissions.

Complete Flow

Step 1

User opens Saviynt



Service Provider (Saviynt)

Step 2

Saviynt creates SAML Authentication Request



Browser redirects to Okta

Step 3

Browser sends SAML Request to Okta



Identity Provider (Okta)

Step 4

Okta authenticates the user



Username + Password (+ MFA)

Step 5

Okta generates SAML Assertion



Browser receives SAML Assertion

Step 6

Browser sends SAML Assertion to Saviynt



Saviynt validates the assertion

Step 7

Saviynt creates a user session



Session cookie sent to browser

Step 8
Browser requests the actual resource



Saviynt

Step 9
Saviynt returns the requested page



User sees the Saviynt Dashboard

SP-Initiated vs IdP-Initiated Flow

Your diagram marks both flows:

SP-Initiated Flow (Starts at Step 2)

The user starts by accessing **Saviynt**.

User



Saviynt



Okta



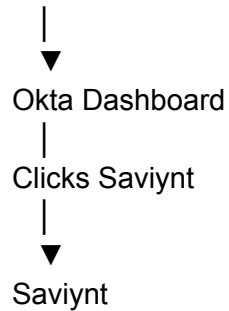
Saviynt Dashboard

This is the most common SAML flow.

IdP-Initiated Flow (Starts at Step 4)

The user starts by logging into **Okta**.

User



In this case, Okta immediately sends the SAML Assertion to Saviynt without Saviynt first sending a SAML Authentication Request.

Interview Explanation (2 Minutes)

In SAML authentication, there are three components: the **Identity Provider (IdP)**, the **Service Provider (SP)**, and the **user's browser**. The user first tries to access the Service Provider, such as Saviynt. Since Saviynt relies on SSO, it redirects the browser to the Identity Provider, such as Okta, with a SAML Authentication Request. The browser forwards this request to Okta. Okta authenticates the user using credentials or MFA. After successful authentication, Okta creates a digitally signed SAML Assertion containing user identity information and sends it to the browser. The browser forwards the assertion to Saviynt. Saviynt validates the assertion, creates a user session, and returns a session cookie. The browser then requests the required resource, and Saviynt responds with the requested page, allowing the user to access the application without entering credentials again.

What is Zero-Day Provisioning?

Definition:

Zero-Day Provisioning means providing all the required accounts, applications, roles, and access to a user on their very first day of joining the organization.

In simple words:

When an employee logs in on Day 1, everything they need is already available.

Why is it called "Zero-Day"?

- **Day 0 (Before the employee starts work):** HR creates the employee record.
- **Day 1:** Employee joins the company.

The employee already has all required access **before or immediately when they start working.**

That's why it's called **Zero-Day Provisioning.**

Example

Suppose Rahul joins ABC Company on **1st July.**

Without Zero-Day Provisioning

Day 1

-  No email
-  No Teams
-  No laptop login
-  No Jira
-  No VPN

Rahul cannot work.

Manager raises tickets.

IT creates accounts manually.

It may take **2–3 days**.

With Zero-Day Provisioning

Before Rahul joins:

HR creates Rahul in Workday.

↓

Saviynt imports Rahul.

↓

Saviynt assigns:

- ☒ Active Directory Account
- ☒ Outlook
- ☒ Teams
- ☒ VPN
- ☒ Jira
- ☒ Laptop Login
- ☒ Default SAV Role

On **Day 1**, Rahul logs in and starts working immediately.

Module-2 : SAV Roles (Saviynt Roles)

What are SAV Roles?

Definition

A **SAV Role** is a role used **inside the Saviynt Identity Cloud** to control **what a user can do within the Saviynt application**.

SAV Roles define the menus, features, pages, APIs, and administrative functions that a user can access after logging into Saviynt.

Simple Definition

SAV Roles determine a user's permissions inside the Saviynt application itself.

Think of a SAV Role as the "job role" of a user within Saviynt.

Why are SAV Roles Needed?

Every user should not have full administrative access.

Different users perform different responsibilities.

For example:

- Helpdesk users reset passwords.
- Managers approve access requests.
- Compliance Managers run certification campaigns.
- Application Owners manage applications.
- System Administrators configure Saviynt.

Each user should only see the functions required for their job.

SAV Roles enforce this principle.

Purpose of SAV Roles

SAV Roles are used to:

- Restrict access to Saviynt features.
 - Provide Role-Based Access Control (RBAC).
 - Improve security.
 - Prevent accidental changes.
 - Reduce insider threats.
 - Follow the Principle of Least Privilege.
 - Improve audit compliance.
-

Internal Working of SAV Roles

Whenever a user logs into Saviynt, the following process occurs:

Step 1

User is authenticated.

(Authentication is performed using Native Login or SSO.)

↓

Step 2

Saviynt identifies the logged-in user.

↓

Step 3

Saviynt checks which SAV Role(s) are assigned.

↓

Step 4

Permissions from those SAV Roles are loaded.

↓

Step 5

Menus, pages, APIs, buttons, and features become available according to the assigned permissions.



Step 6

The user can perform only those actions allowed by the SAV Role.

If a permission is not included in the SAV Role, the feature is hidden or access is denied.

SAV Role vs Business Role

This is one of the most frequently asked interview questions.

SAV Role	Business Role
Controls access inside Saviynt	Controls access to business applications
Used for Saviynt UI and administration	Used for applications like SAP, Active Directory, Salesforce
Assigned to Saviynt users	Assigned to employees
Controls menus and features	Controls application entitlements
Example: ROLE_ADMIN	Example: Finance Role

Example

Rahul logs into Saviynt.

His SAV Role is:

ROLE_MANAGER

This allows him to:

- Request access
- Approve requests
- Review certifications

Rahul also has a Business Role:

Finance Analyst

That Business Role grants:

- SAP Access
- Oracle Finance
- Shared Drive Access

Notice that these are completely different purposes.

RBAC (Role-Based Access Control)

Definition

Role-Based Access Control (RBAC) is a security model where permissions are assigned to roles instead of individual users.

Users receive permissions by being assigned a role.

Instead of granting permissions one by one, assign a role that already contains the required permissions.

Why RBAC?

Without RBAC:

- Permissions must be assigned individually.
- High chance of mistakes.
- Difficult to audit.

With RBAC:

- Easier administration.
- Better security.
- Standardized permissions.
- Easier compliance.

Example of RBAC

Finance Manager Role

Contains:

- Approve Requests
- View Reports
- Review Certifications

Assign the role.

No need to assign every permission separately.

Persona-Based Access

Saviynt provides access based on the user's **persona** (job function).

Different personas require different permissions.

Common personas include:

System Administrator

Responsibilities

- Configure Saviynt
- Manage connectors
- Configure SSO
- Manage certificates
- Create SAV Roles
- Manage users

Highest level of access.

Helpdesk Administrator

Responsibilities

- Unlock users
- Reset passwords
- Assist end users

Cannot perform full system administration.

Manager

Responsibilities

- Request access for employees
- Approve access requests
- Review certification campaigns
- Set delegates

Does not configure the platform.

Compliance Manager

Responsibilities

- Create certification campaigns
 - Run audits
 - Monitor SoD violations
 - Review compliance reports
 - Report risks to the CISO
-

Role Owner

Responsibilities

- Manage Business Roles
 - Review role assignments
 - Approve role requests
-

Out-of-the-Box (OOTB) SAV Roles

Saviynt provides predefined SAV Roles.

These are called **Out-of-the-Box (OOTB) Roles**.

Examples include administrative and manager roles that are ready to use.

Advantages

- Already tested.
 - Secure defaults.
 - No configuration required.
 - Easy deployment.
-

Important Characteristics

OOTB Roles:

- Cannot be modified directly.
- Feature permissions cannot be edited.
- Web Service permissions cannot be edited.
- Display an OOTB icon in the SAV Roles page.

If changes are required:

Clone the role.

Modify the cloned copy.

Custom SAV Roles

Organizations usually need permissions that differ from the default roles.

Saviynt allows creation of Custom SAV Roles.

Custom Roles can:

- Be created from scratch.
 - Be cloned from existing roles.
 - Be modified as required.
-

Why Create Custom Roles?

Example

An Application Administrator should manage applications only.

They should NOT:

- Configure SSO
- Delete users
- Manage certificates

A custom role provides only the required permissions.

Naming Standards

Saviynt recommends the following naming convention:

Always prefix with:

ROLE_

Examples

ROLE_HELPDESK

ROLE_MANAGER

ROLE_FINANCE_ADMIN

ROLE_AUDITOR

Use:

- Uppercase letters
- Meaningful names

- Consistent naming

Avoid:

AdminRole

Role1

TestRole

Feature Access

Feature Access controls which pages and menus appear inside Saviynt.

Examples

Users

Applications

Connections

Reports

Analytics

Certificates

SSO Configuration

Identity Warehouse

Access Requests

If Feature Access is not granted:

The menu is hidden.

Web Service Access

Some Saviynt operations are performed using REST APIs.

Web Service Access determines which APIs a role can use.

Examples

User APIs

Role APIs

Connection APIs

Request APIs

Certification APIs

Without permission:

API requests fail.

API Permissions

Applications integrate with Saviynt through APIs.

API permissions determine:

- Which APIs are allowed.
- Read access.
- Write access.
- Administrative API access.

Example

A Service Account may receive:

User Read API

But NOT:

Delete User API

Access Request Customization Using SAV Roles

SAV Roles control who a user can request access for.

Options include:

Request for Self

User requests access only for themselves.

Request for Self and Direct Reports

Managers can request access for employees reporting directly to them.

Request for Self, Direct Reports, and Two Levels Below

Managers can request access for:

- Direct reports
- Employees reporting to their direct reports

Useful for senior managers.

Request for All

Usually assigned only to administrators.

Allows requesting access for any user.

Role Hierarchy

Organizations often create role hierarchies.

Example

ROLE_ADMIN

↓

ROLE_SECURITY_ADMIN

↓

ROLE_APPLICATION_ADMIN

↓

ROLE_MANAGER

↓

ROLE_HELPDESK

↓

ROLE_ENDUSER

Higher roles usually inherit more responsibilities.

The hierarchy should always follow the Principle of Least Privilege.

Why ROLE_ADMIN Should Never Be Modified

ROLE_ADMIN is the most powerful built-in role.

It contains almost every permission.

Modifying it may:

- Break upgrades.
- Cause unexpected behavior.
- Affect security.
- Make troubleshooting difficult.

Best Practice

Never edit ROLE_ADMIN.

Instead:

Clone it.

Example

ROLE_ADMIN_CUSTOM

Modify the cloned role.

Why ROLE_ADMIN Should Be Assigned to Very Few Users

ROLE_ADMIN can:

- Delete users.
- Delete applications.
- Configure SSO.
- Manage certificates.
- Modify workflows.
- Create roles.
- Run imports.
- Configure connectors.

If assigned to many users:

- Security risk increases.
- Insider threats increase.
- Audit findings increase.
- Accidental changes become more likely.

Only trusted platform administrators should have ROLE_ADMIN.

Service Account Roles

Service Accounts should NEVER use ROLE_ADMIN.

Instead:

Create a dedicated API role.

Example

ROLE_API_SERVICE

Grant only:

- Required API permissions.
- Required web services.
- No UI administration.

Reason

If the service account is compromised, the attacker receives only minimal permissions instead of full administrator access.

Real-Time Example 1

Employee: Rahul

SAV Role:

ROLE_MANAGER

Can:

- Request Access
- Approve Requests
- View Certifications

Cannot:

- Configure SSO
 - Manage Certificates
 - Create Connectors
-

Real-Time Example 2

Employee: Priya

SAV Role:

ROLE_HELPDESK

Can:

- Unlock Users
- Reset Passwords
- Assist End Users

Cannot:

- Delete Applications
 - Configure Identity Warehouse
 - Modify Roles
-

Real-Time Example 3

API Service Account

Role:

ROLE_API_SERVICE

Permissions:

- Read Users API
- Create Access Requests API

Cannot:

- Delete Users
 - Configure SSO
 - Access the Admin UI
-

Best Practices

- Follow the Principle of Least Privilege.
 - Use OOTB Roles whenever possible.
 - Clone OOTB roles instead of modifying them.
 - Prefix custom roles with **ROLE_**.
 - Use uppercase role names.
 - Regularly review role assignments.
 - Assign ROLE_ADMIN only to a small number of trusted administrators.
 - Create separate roles for service accounts.
 - Grant only the API permissions that are required.
 - Periodically audit SAV Role assignments.
-

Interview Questions

Q1. What is a SAV Role?

A SAV Role controls what a user can access and perform inside the Saviynt Identity Cloud.

Q2. What is the difference between a SAV Role and a Business Role?

A SAV Role controls access to Saviynt features and administration, while a Business Role grants application access and entitlements in target systems.

Q3. Can Out-of-the-Box SAV Roles be modified?

No. OOTB roles cannot be modified directly. If changes are required, clone the role and modify the cloned copy.

Q4. Why should ROLE_ADMIN not be modified?

Because it is a built-in role used by Saviynt. Modifying it can affect upgrades, security, and platform stability. The recommended approach is to clone ROLE_ADMIN and customize the copy.

Q5. Why should service accounts not use **ROLE_ADMIN**?

Service accounts should have only the minimum permissions required for their APIs or integrations. Assigning **ROLE_ADMIN** violates the Principle of Least Privilege and increases security risk.

Quick Revision

- SAV Roles control permissions inside Saviynt.
- Business Roles control access to business applications.
- RBAC assigns permissions through roles instead of directly to users.
- Persona-Based Access provides permissions based on job function.
- OOTB roles are predefined and cannot be edited.
- Custom roles are created by cloning or building new roles.
- Feature Access controls UI menus and pages.
- Web Service Access controls API usage.
- Follow the **ROLE_** naming convention using uppercase letters.
- Never modify **ROLE_ADMIN**; clone it instead.
- Assign **ROLE_ADMIN** only to a small number of trusted administrators.
- Create dedicated, least-privilege roles for service accounts and APIs.

Module-3: Certificate Management

What is Certificate Management?

Definition

Certificate Management is the process of **creating, importing, storing, monitoring, renewing, replacing, and deleting digital certificates** used for secure communication between Saviynt and external systems.

Certificates establish trust between two systems and ensure that data exchanged between them is encrypted and cannot be modified by attackers.

In Saviynt, certificates are centrally managed through the **Certificate Management** section under **Admin Functions**. They can also be managed from the **Connection Parameters** page for individual connections.

Why is Certificate Management Important?

Saviynt communicates with many external systems, such as:

- Active Directory
- Azure AD
- Okta
- SAP
- Salesforce
- LDAP
- AWS
- REST APIs
- SOAP Services

Since sensitive information such as usernames, passwords, tokens, and identity data is exchanged, communication must be secure.

Certificates provide this security.

Without certificates:

- Data can be intercepted.
 - Systems cannot verify each other's identity.
 - SSO will fail.
 - API calls may fail.
 - HTTPS communication becomes insecure.
-

What is SSL?

Definition

SSL (Secure Sockets Layer) is a security protocol used to encrypt communication between two systems.

Although SSL is commonly used as the term, modern systems actually use **TLS (Transport Layer Security)**, which is the newer and more secure version.

In everyday conversations, people still refer to TLS certificates as **SSL Certificates**.

Why is SSL Needed?

SSL protects communication from attackers.

It provides:

- Encryption
 - Authentication
 - Data Integrity
-

Example

Without SSL

Rahul logs into Saviynt.

Username and password travel across the network in plain text.

A hacker monitoring the network can steal them.

With SSL

The same information is encrypted.

Even if intercepted, the attacker cannot read it.

What is Encryption?

Encryption converts readable data (Plain Text) into unreadable data (Cipher Text).

Only someone possessing the correct decryption key can convert it back into readable information.

Example

Plain Text

Password123

↓

Encrypted

A7F3B9XQ92M...

Attackers cannot understand encrypted data.

Why are Certificates Needed?

Certificates are required because encryption alone is not enough.

The receiving system must also verify:

"Am I really communicating with the genuine server?"

Certificates answer that question.

Certificates provide:

- Identity Verification
 - Encryption
 - Secure Communication
 - Trust
 - Digital Signature Validation
-

Real Example

Suppose Saviynt connects to Okta.

Before sending authentication information:

Saviynt checks:

"Is this really Okta?"

Okta proves its identity using its certificate.

Only after successful validation is communication established.

Public Key Cryptography

Certificates work using **Asymmetric Encryption**.

Two keys are generated.

- Public Key
- Private Key

Both are mathematically related.

Public Key

Definition

The Public Key is shared with everyone.

It is used to:

- Encrypt data
- Verify digital signatures

Anyone can possess the Public Key.

It is not confidential.

Example

Saviynt receives Okta's Public Key.

Saviynt uses it to verify signed SAML Responses.

Private Key

Definition

The Private Key is secret.

It should never be shared.

It is used to:

- Decrypt data
- Create Digital Signatures

Only the certificate owner possesses the Private Key.

Example

Okta signs every SAML Assertion using its Private Key.

Saviynt verifies that signature using Okta's Public Key.

Public Key vs Private Key

Public Key	Private Key
Shared openly	Kept secret
Encrypts data	Decrypts data
Verifies signatures	Creates signatures
Can be distributed	Never shared

Easy way to remember:

Public Key = Lock

Private Key = Key that opens the lock

What is an X.509 Certificate?

Definition

An X.509 Certificate is a standardized digital certificate used to verify the identity of a server, application, or organization.

Saviynt uses X.509 certificates for:

- SAML SSO
 - HTTPS
 - REST Connectors
 - SOAP Connectors
 - LDAP
 - Active Directory
 - API Integrations
-

Information Stored Inside an X.509 Certificate

An X.509 certificate contains:

- Certificate Owner
 - Organization Name
 - Domain Name
 - Public Key
 - Certificate Serial Number
 - Valid From Date
 - Expiry Date
 - Issuing Certificate Authority (CA)
 - Digital Signature
 - Encryption Algorithm
-

Certificate Authority (CA)

A Certificate Authority is a trusted organization that issues digital certificates.

Examples:

- DigiCert
- GlobalSign
- Sectigo
- Entrust

The CA verifies the owner's identity before issuing a certificate.

Because the CA is trusted, systems trust certificates issued by that CA.

Self-Signed Certificate vs CA-Signed Certificate

Self-Signed Certificate

- Created by the organization itself.
 - Useful for development or testing.
 - Not trusted by default.
-

CA-Signed Certificate

- Issued by a trusted Certificate Authority.
 - Recommended for production environments.
 - Trusted by browsers and applications.
-

Trust Store

Definition

A Trust Store is a secure repository where trusted certificates are stored.

Saviynt uses the Trust Store to determine whether another system is trustworthy.

Example

Saviynt receives a certificate from Okta.

Saviynt checks:

"Does this certificate exist in my Trust Store?"

If Yes:

Connection continues.

If No:

Connection is rejected.

Certificate Lifecycle

Certificates have a lifecycle.

They are not permanent.

Typical lifecycle:

Step 1

Generate Key Pair

↓

Step 2

Create Certificate Request (CSR)

↓

Step 3

Certificate Authority issues certificate

↓

Step 4

Import certificate into Saviynt



Step 5

Use certificate for communication



Step 6

Monitor expiration



Step 7

Renew certificate



Step 8

Replace old certificate



Step 9

Delete expired certificate

Expiry Notifications

Certificates expire after a fixed period.

Examples:

- 1 Year
- 2 Years

- 5 Years

If certificates expire:

- SSO stops working.
- HTTPS connections fail.
- API integrations stop.
- LDAP authentication fails.

Saviynt automatically generates expiry notifications before expiration.

Administrators receive alerts so certificates can be renewed in advance.

Certificate Mapping

Every certificate must be associated with the correct connection.

Examples:

Okta Connection

↓

Okta Certificate

Salesforce Connection

↓

Salesforce Certificate

LDAP Connection

↓

LDAP Certificate

Saviynt automatically maps certificates imported through the **Connection Parameters** page to their respective connections.

This mapping enables accurate validation and expiry monitoring.

Connection Parameters Page

The Connection Parameters page allows administrators to:

- Import certificates
- Associate certificates with specific connections
- View certificate details
- Update certificates
- Replace expired certificates

Certificates imported from this page are automatically linked to the appropriate connection.

Certificate Lifecycle Management in Saviynt

Saviynt provides centralized lifecycle management.

Administrators can:

- Import certificates
- Replace certificates
- Update certificates
- Delete certificates
- Monitor expiration dates
- Track certificate status
- View associated connections

Everything is managed from one interface.

Audit Trail

Every certificate action is recorded.

Examples:

- Import
- Replace

- Delete
- Renew
- Update

The Audit Trail stores:

- Who performed the action
- What action was performed
- Date and Time
- Previous Value
- New Value

This supports:

- Compliance
 - Security investigations
 - Regulatory audits
-

Real-Time Example

Company:

ABC Bank

Saviynt uses Okta for SSO.

The Okta certificate expires next week.

Saviynt automatically sends an expiry notification.

Administrator:

- Generates a new certificate.
- Uploads it into Saviynt.
- Updates the Okta SAML configuration.
- Tests SSO.
- Removes the old certificate.

Employees continue logging in without interruption.

What Happens if a Certificate Expires?

Possible issues include:

- Users cannot log in through SSO.
- REST API calls fail.
- HTTPS communication fails.
- Connector synchronization stops.
- Authentication errors occur.
- Browser security warnings appear.

This is why expiry monitoring is critical.

Certificate Management in Saviynt

Certificates are managed from:

Admin Functions → Certificate Management

Administrators can:

- View certificates
- Import new certificates
- Replace certificates
- Delete certificates
- Monitor expiration
- Review audit logs

The Connection Parameters page can also be used to manage and map certificates.

Best Practices

- Always use CA-signed certificates in production.
- Replace certificates before they expire.
- Monitor expiry notifications regularly.
- Protect Private Keys; never share them.
- Store certificates securely in the Trust Store.
- Remove expired or unused certificates.

- Test new certificates in a lower environment before production.
 - Keep certificate mappings accurate.
 - Regularly review the Audit Trail.
 - Use strong encryption algorithms and follow organizational security policies.
-

Interview Questions

Q1. Why are certificates required in Saviynt?

Certificates establish trust, enable encryption, verify server identity, and secure communication between Saviynt and external systems such as Okta, LDAP, REST APIs, and other target applications.

Q2. What is an X.509 certificate?

An X.509 certificate is a standard digital certificate containing a public key and identity information. It is used for authentication, encryption, and digital signatures.

Q3. What is the difference between a Public Key and a Private Key?

The Public Key is shared openly and is used to encrypt data or verify digital signatures. The Private Key is kept secret and is used to decrypt data or create digital signatures.

Q4. What is a Trust Store?

A Trust Store is a secure repository of trusted certificates that Saviynt uses to validate the identity of external systems.

Q5. What happens if an SSL certificate expires?

SSO, HTTPS communication, API integrations, and connector communications may fail. Users may receive authentication or browser trust errors.

Q6. Why does Saviynt generate certificate expiry notifications?

To alert administrators before certificates expire so they can renew and replace them without causing service interruptions.

Quick Revision

- SSL/TLS secures communication through encryption.
- Certificates verify the identity of systems and establish trust.
- Public Key encrypts data and verifies signatures.
- Private Key decrypts data and creates signatures.
- X.509 is the standard certificate format used in Saviynt.
- Trust Store contains trusted certificates.
- Certificate Lifecycle includes generation, issuance, import, monitoring, renewal, replacement, and deletion.
- Expiry Notifications prevent outages caused by expired certificates.
- Certificates imported through the Connection Parameters page are automatically mapped to their connections.
- Audit Trail records every certificate-related action for compliance and troubleshooting.

Module - 3: Best Practices and Troubleshooting

Best practices are recommended methods for configuring and maintaining Saviynt securely and efficiently. Following these practices helps improve security, avoid production issues, simplify audits, and make future upgrades easier.

1. Why Should "Sign Assertion" Be Enabled?

What is Sign Assertion?

A **SAML Assertion** is an XML document sent by the Identity Provider (IdP) to the Service Provider (SP) after successful user authentication.

It contains information such as:

- Username
- Email
- Groups
- Roles
- Authentication Status
- Timestamp

When **Sign Assertion = Yes**, the Identity Provider digitally signs the SAML Assertion using its private key.

Saviynt verifies this signature using the IdP's public certificate.

Why Should It Be Enabled?

Enabling Sign Assertion ensures:

Authentication Integrity

The assertion has not been modified after it was created.

Authentication Authenticity

The assertion was really generated by the trusted Identity Provider.

Protection Against Tampering

No attacker can change:

- Username
- Roles
- Groups
- Email
- Permissions

during transmission.

Prevents Identity Spoofing

Attackers cannot create fake SAML Responses pretending to be legitimate users.

Ensures Compliance

Many security standards require signed authentication messages.

Examples:

- ISO 27001
 - SOC 2
 - PCI DSS
-

What Happens if Sign Assertion is Disabled?

Possible risks:

- SAML messages can be modified.
 - Fake assertions can be injected.
 - User identities can be impersonated.
 - Unauthorized users may gain access.
 - Audit failures may occur.
-

Real Example

Without Sign Assertion

Attacker intercepts a SAML Response.

Changes:

Role = Employee

↓

Role = Administrator

Saviynt cannot verify the modification.

Access may be granted incorrectly.

With Sign Assertion

The attacker changes the message.

Digital Signature becomes invalid.

Saviynt rejects the authentication immediately.

Interview Question

Why does Saviynt recommend enabling Sign Assertion?

Because it guarantees message integrity, verifies the sender's identity, prevents tampering, and protects against SAML spoofing attacks.

2. What Happens if ROLE_ADMIN is Assigned to Everyone?

What is ROLE_ADMIN?

ROLE_ADMIN is the highest privileged Out-of-the-Box (OOTB) SAV Role in Saviynt.

It provides access to almost every administrative function.

Examples include:

- User Management
- Role Management
- SSO Configuration
- Connection Management
- Certificate Management
- Analytics
- Workflows
- API Management
- Identity Warehouse
- Import Jobs

Why Should ROLE_ADMIN Be Restricted?

Only a few trusted platform administrators should have ROLE_ADMIN.

This follows the **Principle of Least Privilege**.

Problems if Everyone Has ROLE_ADMIN

Security Risk

Any employee could perform critical administrative actions.

Insider Threat

A disgruntled employee could:

- Delete applications
 - Disable connectors
 - Modify workflows
 - Remove users
 - Change SSO settings
-

Human Errors

Someone could accidentally:

- Delete roles
- Modify approvals
- Change certifications

These mistakes can affect the entire organization.

Audit Issues

Auditors expect privileged access to be limited.

Giving everyone ROLE_ADMIN violates compliance requirements.

Difficult Troubleshooting

When many users have administrator rights, it becomes difficult to identify who made a particular configuration change.

Best Practice

Assign ROLE_ADMIN only to:

- Platform Administrators
- Senior IAM Engineers

- Trusted Security Administrators
-

Interview Question

Why should `ROLE_ADMIN` not be assigned to all users?

Because it violates the Principle of Least Privilege, increases security risks, creates audit issues, and allows unauthorized administrative changes.

3. Why Should Service Accounts Never Use `ROLE_ADMIN`?

What is a Service Account?

A Service Account is a non-human account used by applications or integrations.

Examples:

- REST Connector
- SOAP Connector
- API Integration
- Scheduled Jobs
- Provisioning Jobs

These accounts operate automatically without human intervention.

Why is `ROLE_ADMIN` Dangerous for Service Accounts?

If the service account is compromised, the attacker gains full administrator access.

The attacker could:

- Delete users
- Create admin accounts

- Disable MFA
- Change SSO settings
- Delete applications
- Modify connectors

This could compromise the entire identity platform.

Best Practice

Create a dedicated API role.

Example:

ROLE_API_SERVICE

Grant only:

- Read Users API
- Create Access Requests API
- Import Accounts API

Nothing else.

This follows the Principle of Least Privilege.

Real Example

Wrong Configuration

Service Account

↓

ROLE_ADMIN

If compromised:

Entire Saviynt environment is compromised.

Correct Configuration

Service Account

↓

ROLE_API_SERVICE

↓

Only required APIs are accessible.

Even if compromised, the attack is limited.

Interview Question

Why should service accounts never have ROLE_ADMIN?

Because service accounts should have only the permissions required for their integrations. Assigning ROLE_ADMIN creates unnecessary security risks.

4. How to Troubleshoot SSO

When SSO does not work, administrators should troubleshoot methodically instead of changing configurations randomly.

Step 1 – Check User Authentication

Can the user log in directly to the Identity Provider?

Example:

Can the user log into Okta?

If No

Problem is in the Identity Provider.

If Yes

Continue troubleshooting.

Step 2 – Verify Metadata XML

Ensure the correct Metadata XML is imported.

Check:

- Entity ID
 - Single Sign-On URL
 - Logout URL
 - Certificate
 - ACS URL
-

Step 3 – Verify X.509 Certificate

Check:

- Certificate validity
- Expiration date
- Matching certificate between SP and IdP

Expired certificates are one of the most common causes of SSO failure.

Step 4 – Verify Entity ID

Both systems must use exactly the same Entity ID.

Any mismatch causes authentication failure.

Step 5 – Verify ACS URL

ACS

Assertion Consumer Service URL

This is where the IdP sends the SAML Response.

An incorrect ACS URL prevents successful login.

Step 6 – Verify NameID Mapping

Saviynt identifies users using the NameID.

Common mappings include:

- Username
- Email
- Employee ID

If the wrong attribute is sent, Saviynt cannot locate the user.

Step 7 – Check User Exists in Saviynt

The user must already exist in the Saviynt repository.

Otherwise authentication succeeds but authorization fails.

Step 8 – Review Logs

Review:

Saviynt Logs

Identity Provider Logs

Browser Developer Tools

SAML Tracer

These help identify the exact failure point.

Step 9 – Verify System Time

Large time differences between IdP and SP can invalidate SAML tokens.

Always synchronize servers using NTP.

Common SSO Errors

Error	Cause
Invalid Signature	Incorrect or expired certificate
User Not Found	User missing in Saviynt
Invalid Assertion	Modified SAML Response
Metadata Error	Incorrect XML
Entity ID Mismatch	Different Entity IDs
Clock Skew	System time mismatch
NameID Mapping Error	Wrong user attribute

5. Common Certificate Problems

Certificates are frequently responsible for integration failures.

Expired Certificate

Most common problem.

Symptoms:

- SSO failure
- HTTPS failure
- API errors

Solution

Renew the certificate.

Replace it in Saviynt.

Certificate Mismatch

The certificate configured in Saviynt differs from the certificate used by the Identity Provider.

Result

Signature validation fails.

Wrong Certificate Imported

Uploading an incorrect certificate prevents trust from being established.

Always verify:

- Issuer
 - Subject
 - Thumbprint
 - Expiration
-

Missing Private Key

Some certificates require both:

Certificate

-

Private Key

Without the private key, digital signatures cannot be created.

Certificate Not Trusted

Certificate is not present in the Trust Store.

Saviynt rejects communication.

Incorrect Mapping

Certificate is associated with the wrong connection.

Example

Okta Certificate

↓

Mapped to Salesforce Connection

Authentication fails.

Expired Trust Chain

Root CA or Intermediate CA certificates may also expire.

Even if the server certificate is valid, the chain becomes untrusted.

Best Practices for Certificate Management

- Use CA-signed certificates in production.
- Replace certificates before expiration.
- Enable expiry notifications.
- Store certificates securely in the Trust Store.
- Protect private keys.
- Never share private keys.
- Verify certificate mappings after updates.
- Test certificates in lower environments before production.
- Remove unused or expired certificates.

- Review the Audit Trail regularly.
-

Summary

- Enable **Sign Assertion** to ensure SAML responses are authentic and cannot be tampered with.
- Restrict **ROLE_ADMIN** to a very small group of trusted administrators to reduce security and audit risks.
- Never assign **ROLE_ADMIN** to service accounts; instead, create dedicated least-privilege API roles.
- Troubleshoot SSO by validating the IdP login, Metadata XML, X.509 certificates, Entity ID, ACS URL, NameID mapping, user existence, logs, and system time.
- Monitor certificates proactively, renew them before expiration, and keep mappings and trust relationships accurate to avoid outages.